

CS224R Spring 2026 Homework 3

Offline RL

Due 5/8/2026

SUNet ID:

Name:

Collaborators:

By turning in this assignment, I agree by the Stanford honor code and declare that all of this is my own work.

Overview

Goals: In this assignment, you will implement two offline reinforcement learning algorithms: Advantage-Weighted Actor-Critic (AWAC) and Implicit Q-Learning (IQL). You will compare their performance across offline datasets on tasks of different difficulty levels in Antmaze which is a domain from [D4RL](#). You will also study the stitching behavior of offline RL using IQL in the PointMass environment.

Your objectives are as follows:

1. Implement and train AWAC and IQL algorithms for offline reinforcement learning on the Antmaze tasks, and compare their performance.
2. Experiment with key hyperparameters for offline RL algorithms and analyze how they affect performance.
3. Study the stitching behavior of offline RL using IQL on a PointMass task with suboptimal data, and evaluate whether the learned policy can compose better trajectories than those seen in the data.

Submitting the PDF: Fill in your responses in the provided `answer{}` tags. Submit all requested values in tables, and put in all required plots/images as Tex figures. Your PDF should also include all your reasoning and text responses.

Submitting the Code and Experiment Runs: To turn in your code and experiment logs, create a folder that contains the following:

- `csv_data/` folder with all CSV files used for autograding, organized according to the structure described in the Autograder section. Please follow the required directory structure and submission guidelines (number of files per experiment, seed requirements, etc.) as specified there.
- `cs224r/` folder with all the `.py` files, with the same names and directory structure as the original homework repository.

Zip and submit the folder on Gradescope.

Gradescope: Submit both the PDF and the code and experiment runs in the appropriate assignment on Gradescope. An autograder will be provided to evaluate the performance of your policies from the generated tensorboard files.

Use of GPT/Codex/Copilot: For the sake of deeper understanding on implementing imitation learning methods, assistance from generative models to write code for this homework is prohibited.

Autograder

Autograder Submission Instructions (CSV from wandb) For autograding, we provide graders for **both parts of Question 1** (to test your AWAC implementation on two environments) and for the **first two parts of Question 2** (to test your IQL implementation on two environments).

For Question 2, Part 1, you will sweep different IQL expectile hyperparameters. However, for autograder submission, you should **only upload the CSV files corresponding to the best-performing expectile hyperparameter**.

Please download the CSV files **directly from the Eval_AverageReturn plot in your Weights & Biases (wandb) logs**. Specifically, you must export the CSV from the plot named exactly Eval_AverageReturn in the wandb dashboard. **Do not** download CSVs from any other plots or metrics. For each run, use the wandb UI to export the CSV from the Eval_AverageReturn plot corresponding to that specific seed.

Please **download one CSV file per single seed**, rather than combining multiple seeds into one file. This means that for any experiment run with **3 random seeds**, you should have **3 separate CSV files**, one for each seed. Therefore, for each autograded part, your submission must include **exactly 3 CSV files corresponding to the same experiment configuration across the 3 seeds**, all exported from the Eval_AverageReturn plot.

Required CSV File Structure

P1/

```
| - 1/
|   | - awac_umaze_seed1.csv
|   | - awac_umaze_seed2.csv
|   | - awac_umaze_seed3.csv
| - 2/
|   | - awac_medium_maze_seed1.csv
|   | - awac_medium_maze_seed2.csv
|   | - awac_medium_maze_seed3.csv
```

P2/

```
| - 1/  
|   | - iql_umaze_seed1.csv  
|   | - iql_umaze_seed2.csv  
|   | - iql_umaze_seed3.csv  
| - 2/  
    | - iql_medium_maze_seed1.csv  
    | - iql_medium_maze_seed2.csv  
    | - iql_medium_maze_seed3.csv
```

Important Notes

- Each folder (e.g., P1/1/, P2/2/) must contain **exactly 3 CSV files**, one per seed.
- All 3 CSV files within a folder must correspond to the **same experiment setting**.
- For IQL (Question 2, Part 1), ensure the submitted CSVs are from the **best expectile hyperparameter only**.
- Do **not** merge multiple seeds into a single CSV file.

Setup

Codebase: We provide you with offline transitions dataset $\mathcal{D}$. Your goal is to implement the AWAC and IQL reinforcement learning algorithms to learn a policy from this dataset. For each problem, we provide you with the files that you need to complete. Sections that need to be filled have been marked with TODO tags.

Modal Compute Setup: For this assignment, you will complete all sections on Modal instances. **We recommend only using Modal for compute**, as we **do not** support setup on any other platform, such as your own local computer. You may still develop code on your local computer, which we suggest to save compute credits. You may find [this guide](#) helpful on how to set up your code to run on modal.

For set up, create a conda environment that you will use to launch modal with from your local machine:

```
conda create -n cs224r-hw3 python=3.10.19  
conda activate cs224r-hw3
```

For simplicity, for this homework we already provide the entry points needed to run jobs on Modal, so you will not need to configure the full setup yourself. However, before launching any training job, you should complete the basic Modal setup steps below:

1. Install Modal in your conda environment:

```
pip install modal
```

2. Authenticate your account:

```
modal setup
```

3. Redeem the course compute credits using the link provided in the course email at modal.com/credits.

After completing these steps, you should be able to launch the provided training scripts on Modal directly.

Retrieving Results from Modal Container : Training outputs (logs, checkpoints, and evaluation videos) are saved to the `cs224r-hw3-results` Modal Volume, mounted at `/root/hw/data` inside the container. To download all results to your local machine, run:

```
mkdir -p data
modal volume get cs224r-hw3-results / ./data/
```

The command above copies all contents from the Modal Volume into a local `data/` directory.

To download only a specific experiment, run

```
modal volume get cs224r-hw3-results /<exp-name>/ ./data/
```

You can browse the volume without downloading by running:

```
modal volume ls cs224r-hw3-results
modal volume ls cs224r-hw3-results /<exp-name>/
```

Weights and Biases Experiment Tracking : This homework uses Weights and Biases (Wandb) logging to track training stats. You will need a wandb account at <https://wandb.ai/site/>.

Step 1: Create a wandb account.

Go to <https://wandb.ai/site> and sign up for a free account. You can authenticate using your existing GitHub or Google credentials.

Step 2: Install the wandb library.

Install via conda in your terminal (or add it to your `environment.yml`):

```
conda install -c conda-forge wandb
```

This step should be handled automatically when installing the conda environment.

Step 3: Authenticate with your API key.

Run the login command and paste your API key when prompted:

```
wandb login
```

Your API key can be found at <https://wandb.ai/authorize>. The key is saved locally, so you only need to run this once per machine.

If you have any issues with the wandb key not found when running on modal try running the following:

```
modal secret create wandb WANDB_API_KEY=<wandb_key> --force
```

Step 4: Viewing your results.

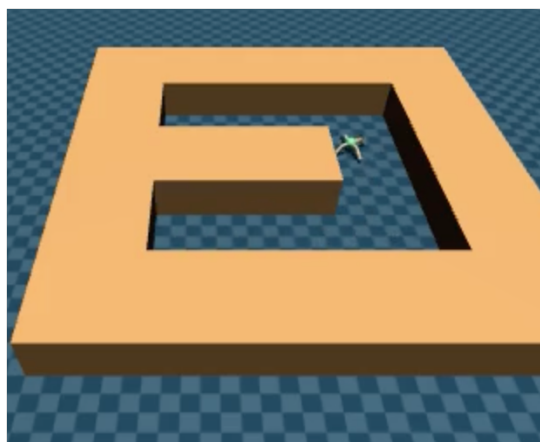
Once training starts, wandb prints a direct URL to your terminal. Open it to see your metrics update in real time or you can also go to <https://wandb.ai/home> and click into the project.

Preliminaries

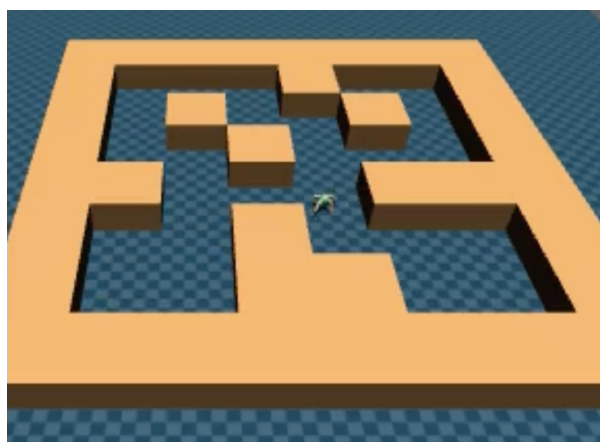
Environments: In this assignment, you will work with two domains: AntMaze and PointMass. The AntMaze domain involves continuous control with a **continuous** action space, while PointMass is a discrete control domain with a **discrete** action space.

In the AntMaze environments, the agent must navigate a maze to reach a goal location. You will experiment with two variants: `antmaze-umaze`, a simpler U-shaped maze where the agent starts near the bottom-left and navigates to the top-left (Figure 1a), and `antmaze-medium`, a more complex maze where the agent starts from the bottom-left and must reach the top-right, requiring longer-horizon planning and making it a harder task. A visualization is shown in Figure 1b

Rewards are sparse: the agent receives a reward of 0 upon reaching the goal and -1 otherwise, which penalizes each non-goal step and encourages the agent to reach the goal as quickly as possible.



(a) Visualization of the `antmaze-umaze` environment.



(b) Visualization of the `antmaze-medium` environment.

Figure 1: Visualization of AntMaze environments

In the PointMass environment, the goal is to navigate a gridworld and reach the goal location. The PointMass environment is visualized in Figure 2. Rewards are sparse: the agent receives -1 at every step and 0 upon reaching the goal. A visualization is in Figure 2

Evaluations: Performance in both AntMaze and PointMass is evaluated using the average return over evaluation episodes. The return R is defined as the sum of rewards collected over an episode. Rewards are in $\{-1, 0\}$, where the agent receives 0 upon reaching the goal and -1 at all other timesteps. Episodes in which the agent fails to reach the goal will therefore have cumulative rewards close to $-\text{episode_length}$, while successful episodes will have higher returns (closer to 0) depending on how quickly the goal is reached.

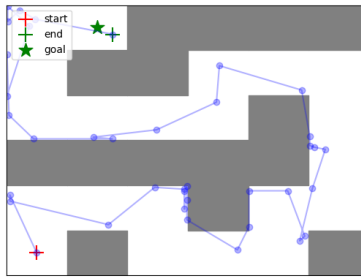


Figure 2: Visualization of the PointMass environment.

Offline datasets: Offline datasets for AntMaze environments will be automatically downloaded when you run the scripts. The datasets will be automatically downloaded to `~/.d4rl/datasets/` at the start of each run, and will be cleared once the container exits after the job finishes.

We also provide an additional dataset, `pointmass_stitching_dataset.npz`, which is specifically designed to study the stitching behavior of offline reinforcement learning in PointMass. This dataset can be found under `offline_datasets/` in your homework code-base.

Problem 1: Advantage-Weighted Actor-Critic [2 points]

In this problem, you will implement the **Advantage-Weighted Actor-Critic** (AWAC) method for offline RL. The objective for updating the actor is an advantage-weighted negative log-likelihood loss function:

$$L_{\pi}(\psi) = -\mathbb{E}_{(s,a) \sim \mathcal{D}} \left[\log \pi_{\psi}(a \mid s) \exp \left(\frac{1}{\lambda} \mathcal{A}^{\pi_k}(s, a) \right) \right] \quad (1)$$

where $\mathcal{D}$ denotes the offline dataset, $\pi_{\psi}(a \mid s)$ is the policy parameterized by ψ , $\mathcal{A}^{\pi_k}(s, a)$ is the advantage function computed using a policy π_k (implemented as the current policy with a stop gradient), and λ is a temperature parameter.

The Q-function is learned using a Temporal Difference (TD) loss:

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{(s,a,s') \sim \mathcal{D}} \left[\left(Q_{\phi}(s, a) - \left(r(s, a) + \gamma \mathbb{E}_{a' \sim \pi_{\psi}(\cdot \mid s')} [Q_{\bar{\phi}}(s', a')] \right) \right)^2 \right] \quad (2)$$

where Q_{ϕ} is the Q-function parameterized by ϕ , $Q_{\bar{\phi}}$ is a target Q-network, γ is the discount factor, and $a' \sim \pi_{\psi}(\cdot \mid s')$ denotes sampling from the current policy.

To reduce overestimation bias in the critic, we use clipped double Q learning. Instead of maintaining a single Q-function, we maintain two Q-networks Q_{ϕ_1} and Q_{ϕ_2} , each with a corresponding target network $Q_{\bar{\phi}_1}$ and $Q_{\bar{\phi}_2}$. The target value is computed using the minimum of the two target Q-functions:

$$y = r(s, a) + \gamma \mathbb{E}_{a' \sim \pi_{\psi}(\cdot \mid s')} \left[\min_{i \in \{1,2\}} Q_{\bar{\phi}_i}(s', a') \right] \quad (3)$$

The critic is then trained by minimizing the TD error for both Q-functions:

$$\mathbb{E}_{(s,a,s') \sim \mathcal{D}} \left[\sum_{i=1}^2 (Q_{\phi_i}(s, a) - y)^2 \right]. \quad (4)$$

1. [1 point] Fill in the TO-DOS in:

- cs224r/policies/MLP_policy.py
- cs224r/critics/awac_critic.py
- cs224r/agents/awac_agent.py

After completing the TODOs, experiment with your implementation by training an AWAC agent on antmaze-umaze task over the dataset antmaze-umaze-v0 using the command below. The expected training time is around 1 hour.

```
modal run --detach modal_train_para.py --algo awac \
--env-name antmaze-umaze-v0 \
--exp-name awac_antmaze_umaze \
--use-wandb
```


Tip: It's easier to run a single seed using the command below while debugging the code. You may also inspect the final evaluation rollout videos saved under the /data directory to qualitatively assess your agent's performance and behavior (see the Setup section for instructions on downloading logs and videos from the Modal Container).

```
modal run --detach modal_train.py --algo awac \
--env-name antmaze-umaze-v0 \
--exp-name awac_antmaze_umaze \
--use-wandb --seed 1
```

Your tasks: Report the **mean and standard deviation across 3 random seeds** of the **final average return** on the dataset antmaze-umaze-v0. Specifically, for each seed, extract the Eval_AverageReturn at the **final checkpoint** from the wandb logs to obtain one scalar value per seed, and then compute the mean and standard deviation over these three values (i.e., aggregate *across seeds*, not within a single run). The provided script automatically launches experiments for all 3 seeds in **parallel**, and both training and evaluation metrics are logged in wandb.

Answer:
Fill in Table 1.

Table 1: AWAC on antmaze-umaze-v0

Eval Average Return	Eval Std. Dev

2. [1 point] Experiment with your implementation by training an AWAC agent on a harder task antmaze-medium over the dataset antmaze-medium-diverse-v0 using the command below. The expected training time is around 1.5 hours.

```
modal run --detach modal_train_para.py --algo awac \
--env-name antmaze-medium-diverse-v0 \
--exp-name awac_antmaze_medium_diverse \
--use-wandb
```

Your tasks: Report the **mean and standard deviation across 3 random seeds** of the **final average return** on the dataset antmaze-medium-diverse-v0. For each seed, use the Eval_AverageReturn at the **final checkpoint**, and compute the statistics over these three values.

Answer:
Fill in Table 2.

Table 2: AWAC on antmaze-medium-diverse-v0

Eval Average Return	Eval Std. Dev

Problem 2: Implicit Q-Learning [5 points]

In this problem, you'll implement the **Implicit Q-Learning** (IQL) method for offline RL. The actor update for IQL incorporates the advantage function similar to the actor update in the AWAC algorithm (see Equation 1).

For critic updates, IQL modifies the critic update to use expectile regression. The expectile ζ of a random variable X is defined as:

$$\arg \min_{m_\zeta} \mathbb{E}_{x \sim X} \left[L_2^\zeta(x - m_\zeta) \right] \quad (5)$$

$$L_2^\zeta(\mu) = |\zeta - \mathbb{1}\{\mu \leq 0\}| \mu^2 \quad (6)$$

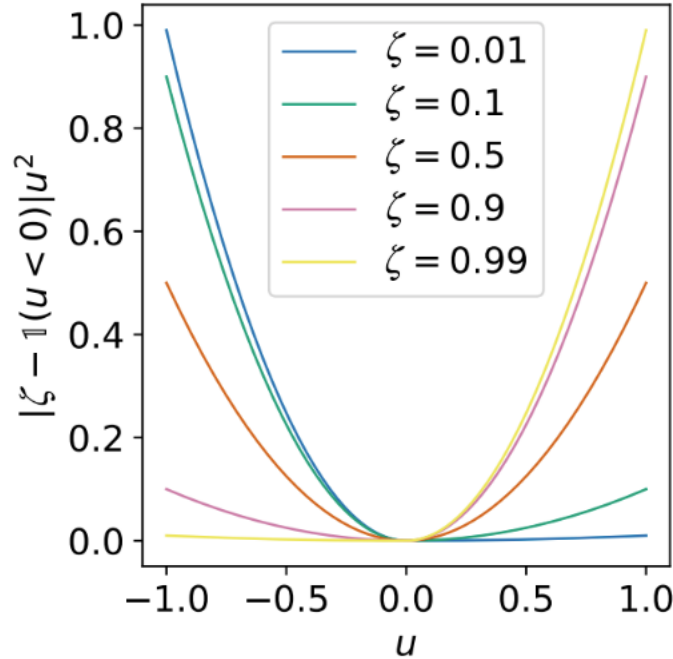


Figure 3: Expectile loss function visualized for different values of ζ .

That is for $\zeta > 0.5$, this asymmetric loss function downweights the contributions of x values smaller than m_ζ , while giving more weights to large values as visualized in Figure 3.

The offline dataset might contain different outcomes from similar states; in standard Q learning, we would want the critic to learn the expected value for any particular state-action pair. IQL, instead of learning the expected value, learns a given percentile. Our goal is to predict an upper expectile of the TD targets that approximate high values of $r(s, a) + \gamma * Q_\theta(s', a')$ which are present in the support of the offline dataset.

To perform this expectile regression, we need a separate parametric value function V_ϕ . Rather than using a single Q-function that requires sampling next-state actions $a' \sim \pi(\cdot |$

s') to compute TD targets, IQL introduces a separate $V_\phi(s')$ that directly estimates the value of the next state from the offline dataset. Finally, the critic is **only** updated with actions that were seen in the offline dataset, and not on any out of distribution (unseen) sampled actions. This leads to the following loss functions:

$$L_V(\phi) = \mathbb{E}_{(s,a) \sim D} \left[L_2^\zeta (Q_{\bar{\theta}}(s, a) - V_\phi(s)) \right], \quad (7)$$

and

$$L_Q(\theta) = \mathbb{E}_{(s,a,s') \sim D} \left[(r(s, a) + \gamma V_\phi(s') - Q_\theta(s, a))^2 \right], \quad (8)$$

where $Q_{\bar{\theta}}$ denotes a target Q-network with parameters $\bar{\theta}$.

1. [2 points] Fill in the TO-DOS in:

- cs224r/critics/iql_critic.py
- cs224r/agents/iql_agent.py

Experiment with $\zeta = 0.2, 0.9$ on the antmaze-umaze-v0 using the command below. The expected training time for each run is around 1 hour.

```
modal run --detach modal_train_para.py --algo iql \
--env-name antmaze-umaze-v0 \
--iql-expectile {enter_your_zeta} \
--exp-name iql_zeta_{enter_your_zeta}_umaze \
--use-wandb
```

Tip: It's easier to run a single seed using the command below while debugging the code.

```
modal run --detach modal_train.py --algo iql \
--env-name antmaze-umaze-v0 \
--iql-expectile {enter_your_zeta} \
--exp-name iql_zeta_{enter_your_zeta}_umaze \
--use-wandb --seed 1
```

Your task:

- (1) Report the **mean and standard deviation across 3 random seeds** of the **final average return** on the dataset antmaze-umaze-v0. For each seed, use the Eval_AverageReturn at the **final checkpoint**, and compute the statistics over these three values.
- (2) In 2-3 sentences, briefly describe which ζ value performs better, and why.

Answer:

(1) Fill in Table 3.

Table 3: IQL on antmaze-umaze-v0

ζ	Eval Average Return	Eval Std. Dev
0.2		
0.9		

(2) Explanation:

2. [2 points] Train an IQL agent on antmaze-medium-diverse-v0 using the **better** ζ value you found in part (1). The expected training time is around 1.5 hours.

```
modal run --detach modal_train_para.py --algo iql \
--env-name antmaze-medium-diverse-v0 \
--iql-expectile {better_zeta} \
--exp-name iql_zeta_{better_zeta}_medium_diverse \
--use-wandb
```

Your task:

- (1) Report the **mean and standard deviation across 3 random seeds** of the **final average return** on the dataset antmaze-medium-diverse-v0. For each seed, use the Eval_AverageReturn at the **final checkpoint**, and compute the statistics over these three values.
- (2) Summarize your results in a table that includes the final average return (averaged across the 3 seeds) for both AWAC and IQL (with the selected ζ) on antmaze-umaze-v0 and antmaze-medium-diverse-v0. Using this table, compare the performance difference between IQL and AWAC across two tasks with different difficulties. In 3 sentences, discuss which algorithm performs better under the harder task (longer horizon and larger map), and explain why in terms of how each algorithm handles out-of-distribution actions and value estimation.

Answer:

(1) Fill in Table 4.

Table 4: IQL on antmaze-medium-diverse-v0

Eval Average Return	Eval Std. Dev

(2) Fill in Table 5 and give the explanation.

Table 5: Final average returns for IQL vs. AWAC across two offline datasets

	AWAC	IQL
antmaze-umaze		
antmaze-medium-diverse		

Explanation:

3. [1 point] In offline reinforcement learning, stitching behavior refers to the ability of an algorithm to combine suboptimal trajectory segments from the dataset to construct a higher-quality policy that was not explicitly observed. In other words, even if the dataset does not contain optimal trajectories, a successful offline RL algorithm can “stitch” together good parts of different trajectories to achieve better performance than any individual trajectory in the dataset.

In this problem, you will investigate whether IQL exhibits stitching behavior on a PointMass task.

You are given a suboptimal dataset `pointmass_stitching_dataset.npz` which with max return -46 and average return -104 .

Train IQL (using the **better** ζ value you found in part (1)) and Filtered Behavioral Cloning (Filtered BC) on this dataset using the provided commands. Filtered BC uses only the top 10% highest-return trajectories. The expected training time for IQL is around 15 minutes and for Filtered BC is around 10 minutes.

```
modal run --detach modal_train_para.py --algo iql \
--env-name PointmassMedium-v0 \
--exp-name iql_zeta_{better_zeta}_stitching \
--iql-expectile {better_zeta} \
--offline-dataset offline_datasets/pointmass_stitching_dataset.npz \
--use-wandb
```

```
modal run --detach modal_train_para.py --algo bc \
--env-name PointmassMedium-v0 \
--exp-name filtered_bc_stitching \
--offline-dataset offline_datasets/pointmass_stitching_dataset.npz \
--filter-top-percent 10 \
--use-wandb
```

Your tasks:

- (1) Report the mean and standard deviation (across **3 different random seeds**) of the average and maximum return for both IQL and Filtered BC.
- (2) Attach the trajectory visualization of the final trajectory generated by the last

checkpoint for both IQL and Filtered BC as a qualitative evaluation. You can find the trajectory visualization figures under the downloaded results directory after retrieving the outputs from the Modal container. See the instruction in the **Retrieving Results from Modal Container** section.

(3) Analyze whether IQL achieves returns higher than -46 , compare its performance with Filtered BC, and analyze whether IQL demonstrates better **stitching behavior**, i.e., whether it can combine suboptimal trajectory segments from the offline dataset to compose a higher-return path that neither trajectory achieves alone.

Answer:

(1) Fill in Table 6.

Table 6: IQL and Filtered BC on PointMass

Algorithm	Average Return $\pm$ Std. Dev	Maximum Return $\pm$ Std. Dev
IQL	$\pm$	$\pm$
Filtered BC	$\pm$	$\pm$

(2) Trajectory visualization figures:

(3) Explanation: